ESCUELA DE INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

SECCIÓN 4

Programación e inteligencia computacional

¿Cómo cambia el oficio de programar cuando parte del código puede generarse automáticamente?

ARTÍCULOS DESTACADOS

- El programador antes y después de los asistentes de IA
- Vibe coding: crear sin comprender completamente
- Comprender, revisar y defender el código generado
- Algoritmos genéticos y creatividad computacional
- Del programador que escribe al programador que dirige y valida



ENTREVISTA · PROFESIONAL

Ing. Luis Fernando Cajas

Desarrollo de software y coordinación técnica



TEMA

Programar con IA: del código al criterio

Ver la entrevista

PERFIL

cambio en la programación?

Luis Fernando Cajas se define ante todo como desarrollador. Acumula cerca de diez años de experiencia en software y actualmente coordina un equipo orientado a análisis de datos, business intelligence y presentación de información. Ese recorrido le permite mirar la IA desde dos perspectivas: como alguien formado en la depuración y la documentación tradicional, y como responsable de equipos que hoy pueden automatizar gran parte del trabajo mecánico.

ANTES

Antes de los asistentes de IA, ¿cómo investigaba, depuraba y refactorizaba código?

El proceso dependía mucho más de documentación oficial, buscadores, foros técnicos, Stack Overflow y videos especializados. Cuando aparecía un error había que reproducirlo, entender exactamente qué ocurría, buscar casos similares, formular hipótesis y probar soluciones una por una. Era más lento, pero obligaba a conocer el código. Para Luis, la IA redujo drásticamente el tiempo de búsqueda; lo que no eliminó fue la necesidad de comprender aquello que se está intentando resolver.

ANTES

¿Qué diferenciaba a un buen programador?

No era escribir más rápido, sino resolver mejor. Había que descomponer una necesidad en problemas pequeños, reconocer restricciones y seleccionar una solución adecuada. Estructuras de datos, algoritmos, bases de datos, arquitectura, sistemas operativos, redes y patrones de diseño eran -y siguen siendo- fundamentos. A ello se sumaba una habilidad esencial: depurar. Seguir la ejecución, plantear hipótesis y comprobar dónde se rompe un sistema forma un criterio que ninguna respuesta generada sustituye automáticamente.

cambió con estas herramientas?

En su trabajo reciente con grandes volúmenes de información, Luis vio el cambio en tareas mecánicas como la generación de consultas SQL. La IA puede acelerar la sintaxis, pero el valor profesional aparece después: revisar índices, pensar en rendimiento, anticipar problemas y optimizar la consulta. El aprendizaje de la transición fue claro: no se traslada el problema completo a la herramienta. La calidad del resultado depende del contexto entregado y de la capacidad de revisar lo que devuelve.

TRANSICIÓN

¿Qué riesgos encuentra en el llamado vibe coding?

Le parece una ventaja enorme porque reduce la barrera de entrada y permite obtener prototipos con rapidez. El riesgo aparece cuando se confunde que una aplicación funcione con que esté bien construida. Puede ejecutar una tarea y aun así contener fallos de seguridad, arquitectura, rendimiento, manejo de errores o mantenimiento. El problema no es utilizar lenguaje natural para construir; es hacerlo sin comprender las decisiones que terminaron dentro del sistema.

ENTREVISTA · CONTINUACIÓN

Ing. Luis Fernando Cajas

Continuación

DESPUÉS

Si la IA produce código que funciona pero el desarrollador no puede explicarlo, ¿debería integrarse?

Para Luis, no de forma directa. Que compile es apenas una condición. Hay que saber qué hace, por qué lo hace, cuáles son sus límites y cómo reaccionará en escenarios no previstos. Si el desarrollador no puede explicar el fragmento tampoco puede garantizar mantenimiento ni responder ante una falla. Antes de integrarlo deberían existir revisión técnica, pruebas de seguridad y arquitectura y, cuando la criticidad lo amerite, una segunda persona que revise la solución.

DESPUÉS

¿Cómo debería revisarse el rendimiento y la mantenibilidad del código generado?

La automatización no elimina áreas de revisión; puede volverlas más importantes. Equipos de QA, revisión técnica y pruebas ayudan a confirmar que la velocidad de generación no se convierta en velocidad para introducir errores. En sistemas con grandes volúmenes de información, por ejemplo, una consulta correcta puede seguir siendo costosa o difícil de mantener. La revisión debe mirar más allá de la salida inmediata y considerar cómo se comportará el componente cuando el sistema crezca.

DESPUÉS

¿Está cambiando el rol del programador?

Sí, hacia un nivel de abstracción mayor. Parte del tiempo que antes se dedicaba a escribir cada línea puede trasladarse a definir restricciones, diseñar soluciones, integrar componentes y validar resultados. Eso no vuelve prescindible al programador; vuelve más importante su criterio. La IA puede hacer a un buen profesional más productivo, pero también permite que un mal criterio produzca errores a una velocidad mucho mayor.

RESPONSABILIDAD

¿Por qué sigue siendo humana la definición de una buena solución?

Porque optimizar no equivale a comprender el propósito. Si una función de aptitud, una restricción o un objetivo están mal definidos, el algoritmo puede encontrar una solución técnicamente óptima para una pregunta equivocada. La máquina puede buscar, generar y optimizar; alguien debe establecer qué significa éxito, qué límites no pueden cruzarse y qué impacto es aceptable. Automatizar la búsqueda de una solución no automatiza la definición del problema.

CIERRE

¿Qué debería dominar un programador aunque una herramienta pueda generar casi cualquier implementación?

Lógica, fundamentos y capacidad de programar sin depender por completo del asistente. Luis insiste en que comprender algoritmos, estructuras, arquitectura y depuración sigue siendo necesario. Sobre esa base pueden añadirse agentes, automatización, inteligencia computacional o nuevas plataformas. La diferencia profesional no estará en negarse a usar IA, sino en poder utilizarla sin entregar a la herramienta el criterio con el que se juzga el sistema construido.

IDEA CENTRAL

Automatizar la búsqueda de una solución no significa automatizar la definición del problema.

ARTÍCULO · ESTUDIANTE

¿Reemplazo o transformación? El impacto de la inteligencia artificial en la programación

Autores: Eduardo Antonio Reyes Pineda; Adriana Lucia Ojeda Rivas

Palabras clave: Inteligencia artificial, programación, asistentes de código, responsabilidad profesional, revisión de código

Introducción:

Durante años ser programador significó dominar varios lenguajes y resolver problemas

escribiendo cada línea de código manualmente. Sin embargo, con la llegada de los asistentes

de IA, esta forma de trabajar ha cambiado rápidamente. Muchos desarrolladores comenzaron

a ver estas herramientas como una amenaza para sus empleos, especialmente porque la IA

puede generar código sin que una persona tenga que escribirlo desde cero.

A pesar de estas preocupaciones, la situación es más compleja que un reemplazo. Este

artículo analiza cómo la IA ha cambiado el trabajo diario de los desarrolladores, dando mayor

importancia a revisar, corregir y validar el código. También se analiza cómo estos cambios

afectan de forma diferente a los perfiles junior y senior. A partir de estudios recientes, se

busca determinar si la IA está reemplazando a los programadores o transformando su trabajo.

Desarrollo:

Tras la llegada de la IA, muchos programadores temieron perder sus empleos. Sin embargo,

el código generado aún presenta errores, por lo que estas herramientas requieren

supervisión humana [1]. Un estudio reciente indica que el 75% de desarrolladores continúa revisando manualmente

el código generado por IA, ya que no se fían al 100% del código que la misma IA les genera

y recurren a revisar minuciosamente cada fragmento de código antes de integrarlo en sus

proyectos [2]. Esto demuestra que, aunque la IA puede generar código, los desarrolladores

todavía necesitan revisarlo antes de integrarlo en sus proyectos.

Casado Martínez y Clarisó Viladrosa comparan este rol con el de un director de orquesta:

quien corrige, ajusta y garantiza el resultado final [1]. Así la IA permite a los desarrolladores

centrarse más en revisar, validar y decidir sobre el código. Sin embargo, la transformación

afecta de forma distinta a senior y junior. Los senior se enfocan más en arquitectura y

revisión, mientras que, según Tobore, un solo ingeniero puede realizar el trabajo de un equipo

completo con IA. Esto plantea dudas sobre las oportunidades para quienes recién comienzan.

Los desarrolladores junior sí se están viendo afectados, ya que las ofertas de empleo

disminuyeron un 49%, y quienes utilizan la IA sin revisar críticamente el código presentan

una comprensión hasta un 17% menor que quienes programan de forma manual [3]. Esto

podría explicar en parte la disminución de contrataciones, al exigir mayor solidez técnica.

Sector	Caídas en oferta
Desarrolladores de Software	49%
Científicos de datos junior	29%
Analistas informáticos	22%

Tabla 1. Caída de ofertas de empleo por sector tras la adopción de IA generativa. Fuente: elaboración propia con datos de Cvejic, Rebelo y Seve.

Sector Caídas en oferta

Como se observa en la Tabla 1, la IA redistribuye responsabilidades según la experiencia,

haciendo que el criterio y la detección de errores sean más importantes que la velocidad de

programación.

Conclusiones:

La inteligencia artificial está transformando la forma en que trabajan los programadores. Su

capacidad para generar código reduce el tiempo dedicado a tareas repetitivas. Sin embargo,

el código generado todavía necesita revisión y supervisión humana. Por esta razón, revisar,

validar y corregir se vuelven habilidades muy importantes. Este cambio también afecta de

manera diferente a los desarrolladores junior y senior.

Los perfiles senior pueden aprovechar la IA para enfocarse más en arquitectura y decisiones

técnicas, mientras los perfiles junior enfrentan mayores dificultades para adquirir experiencia

y acceder a empleos. Esto demuestra que la IA no implica necesariamente el reemplazo de

los programadores, sino un cambio de responsabilidades y habilidades. Por lo tanto, aprender

a utilizar la IA sin depender de ella será clave para los futuros desarrolladores.

Referencias:

[1] Casado Martínez, Carlos, y Ismael Robert Clarisó Viladrosa. "La IA, los programadores

y la nueva revolución del código." UOC, 16 de diciembre de 2025.

https://blogs.uoc.edu/informatica/es/impacto-ia-profesionales-programacion/

[2] Tobore. "How AI Is Reshaping Software Development and the Tech Industry in 2026."

Medium, 25 de febrero de 2026. https://medium.com/@tobore/how-ai-is-reshaping-

software-development-and-the-tech-industry-in-2026-4ec7f7a801df

[3] Cvejic, Ksenija, Luís Daniel Rebelo, y Raphael Seve. "Working with the Machine: How

AI Is Reshaping Software Developer Careers in 2026." Devoteam, 2026.

https://www.devoteam.com/expert-view/ai-impact-software-developer-careers-2026/

ARTÍCULO · ESTUDIANTE

Generado por IA, aprobado por ti: la nueva responsabilidad del programador

Autores: Henry David Quel Santos; Gerardo Leonel Ortiz Tobar

Palabras clave: Código generado por IA, revisión de código, comprensión del código, responsabilidad del desarrollador, asistentes de programación, deuda técnica.

Introducción

Los asistentes de programación basados en inteligencia artificial, como GitHub Copilot,

ChatGPT o Claude, se integraron al flujo diario de miles de desarrolladores y generan funciones

o módulos enteros a partir de una simple instrucción en lenguaje natural. Esta rapidez cambió la

forma de escribir software, pero también introdujo una pregunta incómoda: ¿quién responde

cuando ese código, escrito por una máquina, falla en producción o expone una vulnerabilidad?

Frente a esta realidad surge un principio que debería guiar a cualquier equipo de desarrollo:

ningún fragmento de código generado automáticamente debe integrarse a un sistema sin que una

persona lo comprenda, lo revise y responda por él. Este artículo analiza por qué comprender,

revisar y defender ese código se ha vuelto una competencia tan importante como saber escribirlo,

y qué prácticas permiten sostenerla en equipos reales de ingeniería.

Desarrollo

En un experimento controlado, los desarrolladores que usaron Copilot completaron una tarea de

programación 55% más rápido que quienes no lo usaron (GitHub 2022). Sin embargo, la

velocidad no equivale a confianza: según la Encuesta de Desarrolladores de Stack Overflow,

apenas el 43% confía en la precisión de las herramientas de IA, mientras un 31% se declara

escéptico (Stack Overflow 2024). Esta brecha es el punto de partida de este artículo: usar un

asistente de código no exime de comprenderlo, revisarlo y responder por él.

Comprender un fragmento de código generado significa poder explicar qué hace y qué supuestos

asume sobre los datos de entrada. Los asistentes de código no razonan sobre el problema

particular del equipo: producen una solución estadísticamente plausible a partir de patrones

observados en otros proyectos. Un análisis de más de ciento cincuenta millones de líneas de código encontró que el código descartado o reescrito en las dos semanas siguientes a su creación

se duplicó entre 2021 y 2024, señal de que buena parte de ese código se integra sin comprenderse

a fondo (GitClear 2023).

La revisión de código, o code review, sigue siendo el mecanismo más confiable para detectar

errores antes de producción, y su importancia crece cuando el autor original es un modelo de

lenguaje. No debe limitarse a validar que el código compile: debe verificar que la lógica sea

correcta, que no introduzca vulnerabilidades y que sea mantenible por el resto del equipo. Los

modelos de lenguaje pueden generar salidas inseguras sin un filtro humano, por lo que conviene

tratar cualquier código producido por IA con el mismo escrutinio que el código de una persona

desconocida (OWASP Foundation 2025).

Ningún marco de gestión de riesgos de IA ni norma de la industria exime al desarrollador de

responder por el código que publica, sin importar quién o qué lo haya escrito (NIST 2023). En

una auditoría, un incidente de seguridad o una revisión legal, la pregunta no es «¿quién lo

generó?», sino «¿por qué se aceptó esta implementación?». Defender el código significa poder

justificar cada decisión: por qué se eligió cierta estructura de datos, por qué se manejó así un

error o por qué se confió en una dependencia externa. Esa justificación debe sostenerse incluso

cuando la sugerencia inicial provino de un asistente de IA.

La figura 1 resume este ciclo de responsabilidad: comprender, revisar y defender no son pasos

aislados, sino una práctica continua que se repite cada vez que se acepta una sugerencia generada

automáticamente.

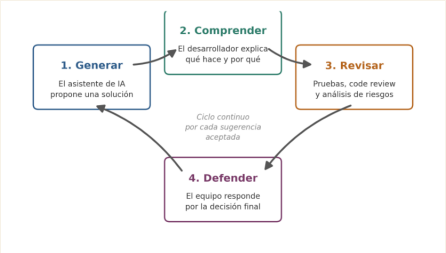


Figura 1. Ciclo de responsabilidad sobre el código generado por IA. Fuente: elaboración propia, con base en OWASP Foundation (2025) y NIST (2023).

Conclusión

Comprender, revisar y defender el código generado por inteligencia artificial no son controles

opcionales: son la condición mínima para adoptar estas herramientas sin heredar una deuda

técnica invisible. La velocidad que ofrecen los asistentes de código solo se traduce en valor real

cuando existe una persona capaz de explicar, corregir y responder por cada línea que termina en

producción.

El reto para los equipos de ingeniería no es prohibir el uso de estas herramientas, sino construir

la disciplina —revisión sistemática, pruebas y documentación de decisiones— que permita

usarlas sin renunciar a la comprensión del sistema que se está construyendo. Esa disciplina, más

que la herramienta misma, es lo que distingue a un equipo capaz de sostener su software en el

tiempo.

Referencias

GitClear. 2023. "Coding on Copilot: 2023 Data Suggests Downward Pressure on Code Quality."

GitClear.

https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_qu

ality

GitHub. 2022. "Research: Quantifying GitHub Copilot’s Impact on Developer Productivity and

Happiness." The GitHub Blog. https://github.blog/news-insights/research/research-quantifying-

github-copilots-impact-on-developer-productivity-and-happiness/

National Institute of Standards and Technology (NIST). 2023. "Artificial Intelligence Risk

Management Framework (AI RMF 1.0)." NIST AI 100-1. https://doi.org/10.6028/NIST.AI.100-1

OWASP Foundation. 2025. "OWASP Top 10 for LLM Applications 2025." OWASP GenAI

Security Project. Consultado el 11 de septiembre de 2026.

https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/

Stack Overflow. 2024. "2024 Stack Overflow Developer Survey: AI." Consultado el 11 de

septiembre de 2026. https://survey.stackoverflow.co/2024/ai

ARTÍCULO · ESTUDIANTE

El fin del desarrollo de software tradicional: programadores en la nueva era de la IA generativa

Autores: Fernando José Vicente Velásquez; Alexander Santiago Ceballos Gil

Palabras clave: Inteligencia Artificial Generativa, Ingeniería de software, Evolución del programador, Desarrollador de software, Calidad de software.

Introducción

Desde la llegada de la Inteligencia Artificial generativa han llegado a surgir muchas incógnitas sobre si

muchas profesiones cambiarían o desaparecerían por completo; el desarrollo de software no fue la

excepción. Observar a la IA generativa completar tareas que antes requerían de mucho tiempo generó

un fuerte impacto. Afortunadamente los desarrolladores de software no van a desaparecer, pero es

necesario que se lleve a cabo una renovación en cuanto a su formación académica.

Es vital transformar tanto el rol como la formación de los desarrolladores. Anteriormente, su trabajo se

limitaba a la codificación manual; sin embargo, aunque la automatización acelera el desarrollo, la falta

de buenas prácticas genera riesgos en los sistemas. Por ello, el objetivo de este artículo es analizar

cómo debe actualizarse este rol en la nueva era de la IA generativa.

El nuevo rol del programador en el ciclo del desarrollo de software

La formación de nuevos programadores debe adaptarse a priorizar la capacidad crítica de

identificación y comprensión de problemas. Es vital que se capaciten en la delegación y orquestación

de recursos técnicos. Esto implica desarrollar un criterio de evaluación ética y de gobernanza.1 La tarea

de los programadores debe consistir en tener el criterio de identificar requerimientos primordiales.

Fallos en la generación no gobernada

Cuando se utilizan herramientas de IA sin respaldo académico y sin verificación técnica se genera el

fenómeno llamado generación no gobernada. 2 Si no se posee control de estos fallos y no se audita el

código activamente, se pueden llegar a manifestar los siguientes modos de fallos recurrentes en el

desarrollo.

1 Mamdouh Alenezi, "Educating the Agentic Engineer: Curricula, Collaboration, and Continuous Learning in the

AI Era," arXiv preprint arXiv:2607.29610 (2026)

2 Mamdouh Alenezi, "Specification-Driven Development as the Foundation of AI-Native Enterprise Software Engineering," arXiv preprint arXiv:2607.16680 (2026), 2–3, https://arxiv.org/html/2607.16680. Como se expone en la Tabla 1, la dependencia excesiva en la IA sin supervisión humana provoca

problemas serios. Se destaca la erosión de la arquitectura y la acumulación de vulnerabilidades.

Modo de fallo	Mecanismo causal
F1: Paradoja productividad fiabilidad	La rapidez y el flujo incentivan la aceptación del comportamiento observado; La práctica de control de calidad es escasa o ausente; Distancia de corrección entre "parece funcionar" y "es correcto"
F2: Erosión arquitectónica	Generador optimiza el contexto del prompt localizado; Las propiedades del sistema de corte transversal son invisibles dentro de una sola generación; Los errores locales se acumulan direccionalmente
F3: Exposición a la seguridad	Generación propensa a la vulnerabilidad agravada por una aceptación humana excesiva; Las debilidades llegan a los repositorios de producción
F4: Acumulación de deuda técnica	El razonamiento detrás del código generado nunca se externaliza en artefactos revisables; Entrelazamiento y deriva entre intención e implementación

Tabla 1. Modos de fallo en la generación de código de IA no gobernada. Fuente: adaptado de Mamdouh Alenezi, “Specification-Driven Development as the Foundation of AI-Native Enterprise Software Engineering“.

AI-Native Enterprise Software Engineering"

Evidencia empírica de la deuda técnica (velocidad vs. calidad)

Un análisis de la firma de analíticas de código GitClear, evaluó más de 153 millones de líneas de

código escritas entre los años 2020 al 2023. Se encontró que, el uso de asistentes de IA se asocia con

un aumento de código duplicado, mientras que la reutilización y el refactorizado de código

estructurado disminuye. 3 La productividad aumenta, la mantenibilidad y comprensión decaen.

Otra investigación causal desarrollada por la Universidad Carnegie Mellon evaluó de forma empírica

la adopción de entornos de desarrollo agénticos. Se determinó que la adopción de agentes de IA genera

un incremento significativo pero temporal en la velocidad de desarrollo de un proyecto. Sin embargo, a

largo plazo, provoca un aumento persistente en las advertencias de análisis estático y en la complejidad

estructural del código.4 Es vital priorizar la revisión humana para mitigar esta deuda técnica.

Soluciones de gobernanza: el desarrollo basado en especificaciones

La arquitectura de esta solución se consolida en el Modelo de Referencia para la Gobernanza de

Especificaciones. Dado que los modelos de lenguaje generan respuestas basadas en probabilidades, es

3 William Harding y Matthew Kloster, Coding on Copilot: 2023 Data Suggests Downward Pressure on Code

Quality, whitepaper (GitClear, 2024), 5,

https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_quality. 4 Hao He et al., "Does AI-Assisted Coding Deliver? A Difference-in-Differences Study of Cursor's Impact on Software Projects," arXiv preprint arXiv:2511.04427 (2025), 4, https://arxiv.org/abs/2511.04427. necesario controlarlos mediante reglas claras. De esta forma, la generación automática de código por

medio de agentes de IA, queda obligada a cumplir estrictamente con tres niveles progresivos de rigor y

control metodológico.5

Conclusiones

Las instituciones de educación superior enfrentan el reto inmediato de reformar radicalmente sus

programas de estudio para formar a las futuras generaciones de desarrolladores de software. Los planes

curriculares deben dejar de medir el éxito de los estudiantes por su capacidad por medio de métricas

que solo se centren en la sintaxis del código y es necesario comenzar a evaluar sus capacidades de

diseño lógico de requerimientos, auditoría de código y gobernanza de la calidad del software. El

Desarrollo Basado en Especificaciones representa la respuesta metodológica e institucional más sólida

para embridar la fluidez probabilística de la IA. Al estructurar el desarrollo de software alrededor de

especificaciones legibles por máquina que sirven como contratos formales, se logra encerrar la

naturaleza estocástica de la IA dentro de límites deterministas de verificación y control. Es así como se

consigue la evolución del programador centrándose en un rol evolucionado.

Referencias bibliográficas

Alenezi, Mamdouh. "Educating the Agentic Engineer: Curricula, Collaboration, and Continuous

Learning in the AI Era." arXiv preprint arXiv:2607.29610, 2026. https://arxiv.org/html/2607.29610.

---, Mamdouh. "Specification-Driven Development as the Foundation of AI-Native Enterprise

Software Engineering." arXiv preprint arXiv:2607.16680, 2026. https://arxiv.org/html/2607.16680.

---, Mamdouh. "The Rise of AI-Native Software Engineering: Implications for Practice, Education,

and the Future Workforce." arXiv preprint arXiv:2606.12986, 2026. https://arxiv.org/abs/2606.12986.

Harding, William, y Matthew Kloster. Coding on Copilot: 2023 Data Suggests Downward Pressure on

Code Quality. Whitepaper, GitClear, 2024.

https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_quality.

He, Hao, Courtney Miller, Shyam Agarwal, Christian Kästner, y Bogdan Vasilescu. "Does AI-Assisted

Coding Deliver? A Difference-in-Differences Study of Cursor's Impact on Software Projects." arXiv

preprint arXiv:2511.04427, 2025. https://arxiv.org/abs/2511.04427.

5 Mamdouh Alenezi, "Educating the Agentic Engineer: Curricula, Collaboration, and Continuous Learning in the AI Era," arXiv preprint arXiv:2607.29610 (2026), 4–5, https://arxiv.org/html/2607.29610.